DEPARTMENT OF INFORMATICS

LUDWIG-MAXIMILANS-UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Detection of Collaterally Covered Goals with CPAchecker's Value Analysis

Author: Jens Hartmann
Supervisor: Prof. Marie-Christine Jakobs
Submission Date: 17.11.2025

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbstständig und nur mit den angegebenen Hilfsmitteln verfasst habe. Alle Passagen, die ich aus einer Literatur oder anderen Quellen übernommen habe, sind eindeutig als Zitate mit Quellenangabe gekennzeichnet. Diese gedruckte Fassung ist ein Ausdruck der eingereichten elektronischen Fassung.

I hereby declare that I have written this work independently and only with the aids specified. All passages that I have taken from literature or other sources have been clearly marked as quotations with reference to the source. This printed copy is a printout of the submitted electronic copy.

Ort/location, Datum/date	Unterschrift/signature
München, 17.11.2025	Jens Hartmann

Tools Used

I used Claude-AI and Deepl to suggest wordings of single sentences and point out spelling mistakes and grammatical errors. I also used Claude-AI for some limited support in writing mathematical latex expressions. I added all suggestions from LLMs manually with full supervision.

Abstract

Software model checkers offer a variety of uses. Among them is the ability to produce quality test suites by generating tests from counterexample traces obtained through reachability analysis. CPAchecker is a versatile verification software, employing value analysis to generate tests. The problem investigated in this thesis centers around the inability of CPAchecker's built-in *Test Case Generation Algorithm* to check whether a newly generated test covers more than one test goal. As a result, test suites generated by CPAchecker tend to include a lot of redundancy, as most tests have the potential to cover multiple goals. The presented solution to this problem is the *extractor component*, an expansion to CPAchecker which introduces multi-goal coverage to its *Test Case Generation Algorithm*. The extractor uses CPAchecker's value analysis to conduct an additional reachability analysis for each test generated. This analysis determines if a test collaterally covers any additional goals, generating abstract successor states that are definitely reachable by the test input. Benchmarks show that the extractor is very successful in covering test goals collaterally. Test suites could be significantly reduced while maintaining the same coverage. Savings in computing resources turned out relatively small but, overall, highlight a slight improvement compared to CPAchecker's original analysis. Integrating the extractor with the CEGAR variant of the value analysis is a promising option for further development.

Contents

Acknowledgments **iii**

Abstract **iv**

1 Introduction **1**

2 Background of Test Generation with Value Analysis **3**
2.1 Programs and their Concrete and Abstract States 4
2.1.1 Control-flow Automaton 4
2.1.2 Concrete States 4
2.1.3 Semi-lattice 6
2.1.4 Relationship between Concrete and Abstract States in the Value Analysis 9
2.2 Configurable Program Analysis (CPA) 10
2.2.1 Components of the CPA 10
2.2.2 CPA Algorithm 10
2.2.3 Abstract Reachability Graph (ARG) 11
2.3 CPAcheckers original Test Case Generation Algotithm (TCGA) 11
2.3.1 Steps of the TCGA 13
2.3.2 Example of running the TCGA 14
2.3.3 Problem with CPAchecker's Test Case Generation Algorithm 16

3 Design of Detection Approach **18**
3.1 Main Parts of the Extractor Component 18
3.1.1 Initialization of the Starting Abstract State 18
3.1.2 Differences between Extraction Analysis and Exploration Analysis . . 19
3.1.3 Coverage Update 20
3.2 Example ARG for Applying Extractor Component 20

4 Implementation of Extractor in CPAchecker **23**
4.1 Realization in CPAchecker's Code 23
4.2 Step by Step Implementation of the Extractor Parts 24
4.2.1 Initialize Starting Abstract State 24
4.2.2 Run Extraction Analysis 25

4.2.3 Coverage Update 25
4.3 Demonstration of Basic Functionality with Examples 25
4.3.1 Demonstration of Collateral Coverage 25
4.3.2 Demonstration of Safe-assume Property 26
4.3.3 Demonstration of Continuation Property 27

5 Evaluation of the Extractor **29**
5.1 Setup 29
5.2 Experiments 30
5.3 Threats to Validity 33

6 Related Work **35**
6.1 Conditional Testing: Test Goal Extractor 35
6.2 Conditional Testing: Cyclic Tester 36
6.3 Multiple Goal Coverage: CPATiger 36

7 Conclusion **37**

Bibliography **39**

1 Introduction

Software testing is an exceedingly important and often underestimated tool for assessing software quality and maintainability. With a very high demand for reliable software, the area of automatic test input generation tools attracts increasing attention. Software testing is about determining if a program behaves as expected in a specific scenario, for example executing it with concrete variable values and observe if the program behaves as expected for that case. Software verification on the other hand is about proving if a program behaves in a desired way for all scenarios. The program behaviour is logically modelled, and conditions like axioms and proofs are derived from that. Verifiers have been used to create tests from counterexamples since 2004[1][11], aiming at generating test suites that maximize a coverage criterion. One particular class of tools are test-case generators based on model checkers. Our work focuses on one of these called CPAchecker[1][6][3], a verification tool that utilizes reachability analysis to generate test suites.

CPAchecker's configurable program analysis (CPA) is designed to check for specification violations. These are unwanted program states, for example, calling an uninitialized variable. It performs a reachability analysis to determine if specification violations are encountered. CPAchecker's *Test Case Generation Algorithm* takes advantage of this, encoding test goals as specification violations. So the algorithm aims at reaching as many violations as possible. For encoding our task in the specification, we will focus only on the structural property of branch coverage as a coverage criterion. The resulting test goals are a subset of the control-flow edges of the program's *control-flow automaton*. For each conditional statement in the program (like if-statements, for-loops etc.), each possible case should be covered as each of these cases is a test goal. During the analysis, every time a test goal is reached, the analysis is interrupted and a counterexample trace is produced, which describes the path to the test goal. From this path, a test is generated and added to the test suite[2]. Then the analysis is resumed. These steps are repeated iteratively until all goals are covered or time or memory limitations are reached. The result is the finished test suite.

However, this presents a problem: a test goal, i.e. a branch in the program, is encountered, a test is generated from it, but is not checked if that test covers multiple goals. CPAchecker's original *Test Case Generation Algorithm* only has knowledge about the test input covering that one initial test goal, which we will from now on refer to as the *test goal of interest*, and is blind

[1] CPAchecker homepage `https://cpachecker.sosy-lab.org/`

[2] Due to the nature of the algorithm, the path described by the counterexample trace never includes uncovered test goals.

to any additional coverage of that test. This leads to redundancy in the generated tests suite and more tests being generated than are needed. While a single test has the potential to cover multiple test goals, CPAchecker's original *Test Case Generation Algorithm* simply does not investigate this. For a test, only the path from the start of the program towards the test goal is known. There is no information about the further path in the program, as CPAchecker's reachability analysis does not represent an actual program execution. It is not accounted for if the new test covers test goals subsequent to the test goal of interest. Instead, for each goal to be covered, the original *Test Case Generation Algorithm* continues the reachability analysis, but has to perform new counterexample checks, generate a new counterexample trace and generate a new test, all of which we want to minimize. There is no multi-goal coverage.

In this thesis we introduce the *extractor component*, an expansion of CPAchecker's *Test Case Generation Algorithm*. This feature can be activated separately and applies CPAchecker's value analysis[2][4] to determine test goals reachable with the generated test inputs. This additional reachability analysis starts right after the test goal of interest. It takes advantage of CPAchecker's internal abstract states and uses them to initialize its own starting state, saving on computation. This analysis calculates the further path and checks for uncovered goals until it can no longer precisely determine a branch direction anymore or it requires a new input that is not provided by the test input.

The efficiency and effectiveness of the extended CPAchecker was evaluated with the TestComp benchmark set[3]. The CPU time, number of covered test goals and number of generated tests were compared using CPAchecker's test case generation configurations with and without the new extractor active.

In summary, we make the following contributions:

- We expand CPAchecker's original *Test Case Generation Algorithm* with the extractor component, adding the ability to detect collateral coverage of tests.
- We perform an evaluation of CPAchecker's extractor component with the standard TestComp benchmarks.
- The extractor component and benchmark results[4] are publicly available.

[3] `https://gitlab.com/sosy-lab/benchmarking/sv-benchmarks/`
[4] `https://zenodo.org/records/17619986`

2 Background of Test Generation with Value Analysis

CPAchecker is an analysis framework that builds on the CPA concept. It is a verifier that is also capable of generating test suites[5]. The value analysis used for this thesis is defined by CPAchecker, with the goal of improving its efficiency for test generation. In order to understand the value analysis, we first take a look at the underlying configurable program analysis (CPA), specifically the variant *constant propagation without join* described in the *Handbook of Model Checking*[4].

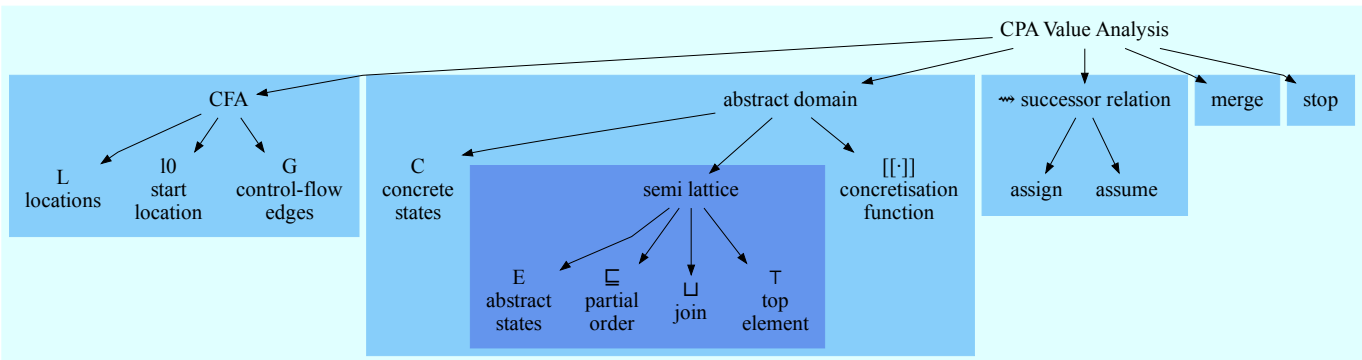

Figure 2.1: Overview of a configurable program analysis $\mathbb{D} = (D, \rightsquigarrow, merge, stop)$ on a CFA. The *abstract domain D* includes the *semi-lattice*, which defines the abstract states that are the basis for the value analysis.

Figure 2.1 gives an overview of the components of a CPA that we will discuss in this chapter in detail. The CPA gets applied on a program in the form of a *control-flow automaton* (CFA). A CFA is a representation of a program as an automaton consisting of program locations, that are represented as nodes, and program operations, represented as so called control-flow edges.

Concrete states represent concrete variable assignments, and the transformation from one concrete state to another is defined by the control-flow edges of the CFA. Next we examine the concept of semi-lattices and the derived abstract states, used to represent abstract variable assignments. The transfer relation serves as the successor function to compute new abstract states. The concretisation function serves as a link between abstract and concrete states. The merge function is used to combine abstract states into new ones. The stop function has the purpose to stop exploration if no new information can be derived from a visited abstract state. All of these components are necessary to define the value analysis as a configurable program analysis. After that we take a look how the analysis is used to generate tests.

2.1 Programs and their Concrete and Abstract States

Our programs use simple C syntax with only *assign* and *assume* operations present. The latter is a term used in model checking for conditional executions. We define an *assume* operation as being one of the following statements: if, if-then-else, ?, while, for, switch. Function calls are not used and variables can only be assigned integer values. Concrete states are an assignment of all variables of the program and the program counter while an abstract state represents many concrete states. Abstract states also feature a half-order, allowing one state to subsume another giving the option to drop a state if another stores the same information.

2.1.1 Control-flow Automaton

A *control-flow automaton* (CFA) represents a program. A CFA (L, l_0, G) consists of the set of program locations L, the starting location l_0 and a set $G \subseteq L \times Ops \times L$ of control-flow edges. Program locations model the program counter, that displays the exact line in the program that the location refers to. Each location refers to exactly one line. The code at that line has not been executed when the corresponding location is reached. Each control-flow edge in G is a transformation of one concrete state (source) into another concrete state (target) when the operation at the program location of the source state is executed. An operation in Ops is either an assignment operation or an assume operation. When representing a CFA in graphical form, like in figure 2.2, program locations are represented as nodes and control-flow edges are represented as edges between those nodes.

2.1.2 Concrete States

Let us start with some definitions: We call the *set of program variables* X and the *program counter* pc (program location). A *concrete state* c is an assignment of $X \cup pc$, so there is an assignment of each variable and the program location to a concrete value. We call the *set of all concrete states* C. A *region* $r \subseteq C$ is a subset of concrete states.

```
1   int y = input();
2   int x = 0;
3   int z = 0;
4
5   if (y == 1) {
6     x = 1;
7   } else {
8     z = 1;
9   }
10  x = 10 / (x - z);
11  return x;
```

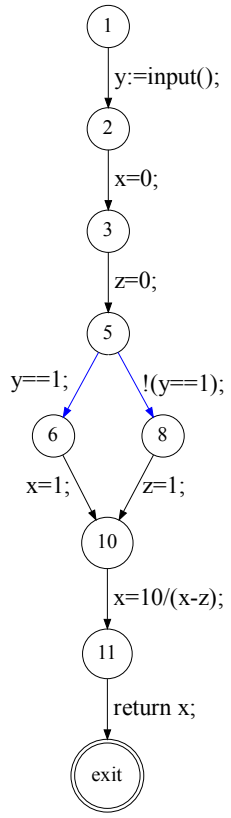

Figure 2.2: Example program with code and CFA. Each node represents the program counter being at a line in the c-code. An edge represents the execution of a line. The blue colored edges are test goals and will be discussed below in section 2.3.

A *control-flow edge* $g \in G$ defines the transition relation $\xrightarrow{g} \subseteq C \times \{g\} \times C$ from one concrete state to another. Note that multiple concrete states can refer to the same program location, sharing the same program counter value. Two states might be transformed by the same control-flow edge, and can become two different or the same program location.

Example Concrete States and Transitions Figure 2.2 shows a example C-program with the corresponding CFA. The program is interesting as there might be a division through zero in line 10, since both x and z could possibly be assigned a new value. The CFA has 9 program locations including the exit state and contains the set of variables $X = [x, y, z]$. The if-statement is represented as a branch in the CFA.

The listing illustrates examples of concrete states and shows how a concrete state is transformed into a new one.

- $(c(pc) = 6, c(y) = 1, c(x) = 0, c(z) = 0)$
  Example concrete state for program location 6. There is one possible concrete state for this location that is reachable in the program, although more can be defined.

- $(c(pc) = 5, c(y) = 0, c(x) = 0, c(z) = 0)$
  Example concrete state for program location 5. Other concrete states for location 5 are also possible, since y could be assigned any value at this location.

- $(c(pc) = 6, c(y) = 1, c(x) = 0, c(z) = 0) \xrightarrow{\text{x=1}} (c(pc) = 10, c(y) = 1, c(x) = 1, c(z) = 0)$
  Only possible transition relation from location 5 to 6. State 6 cannot be reached from any other concrete state.

2.1.3 Semi-lattice

A *semi-lattice* $\mathcal{E} = (E, \sqsubseteq, \sqcup, \top)$ is comprised of a *half-order* $\sqsubseteq \subseteq E \times E$ on a set of *abstract states* E. It also features two additional components: the *join operator* $\sqcup$, which we will not discuss further as it is only important for defining the elements of the semi-lattice, but is not used in the program; and the *top element* $\top$, that is the least upper bound element for all lattice elements, subsumes every other abstract state, meaning $\forall e \in E : e \sqsubseteq \top$. The opposite is the *bottom element* $\bot$, that is subsumed by every other abstract state.

Figure 2.3 shows the main components of the *semi-lattice* and how its elements are instantiated for the value analysis. For this instantiation the abstract states take the form $L \cup \{\top_L\} \times (X \to Z)$ with $Z = \mathbb{Z} \cup \{\top_Z\}$. The first element $L \cup \{\top_L\}$ describes the program location and the second element describes the assignment of each program variable. All variables are mapped either to an integer value, similar to the concrete state assignments, or to the additional element $\top_Z$. A variable assigned $\top_Z$ can be any (possible) integer value. Similarly, the location l can be assigned any program location or $\top_L$, the latter representing all possible program locations.

Half-order The *half-order* on the elements on the semi-lattice is used to compare different abstract states in the value analysis. This way it can be decided if one state is subsumed by another one. *Definition* $\sqsubseteq$:

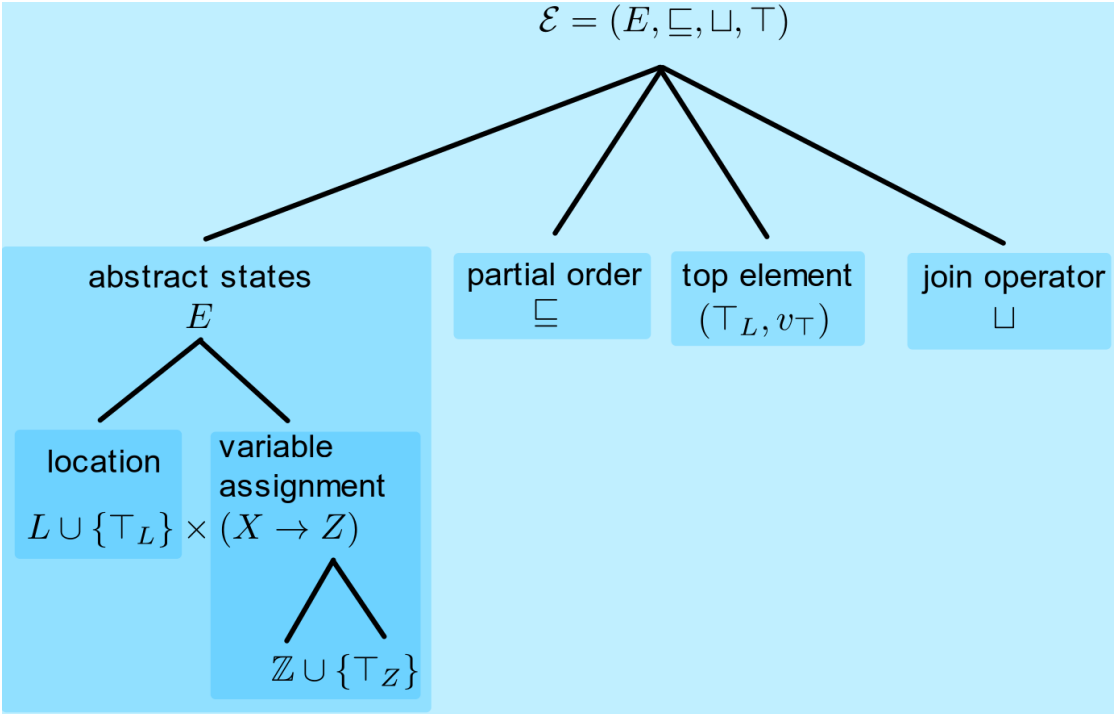

Figure 2.3: The semi-lattice $\mathcal{E} = (E, \sqsubseteq, \sqcup, \top)$ used for the value analysis. Its elements are the *abstract states* E consisting of a program location and variable assignments, the *half-order* $\sqsubseteq$ over these states, the *top element* $\top$, that subsumes every other element and lastly the *join operator*, used for the definition of the half-order.

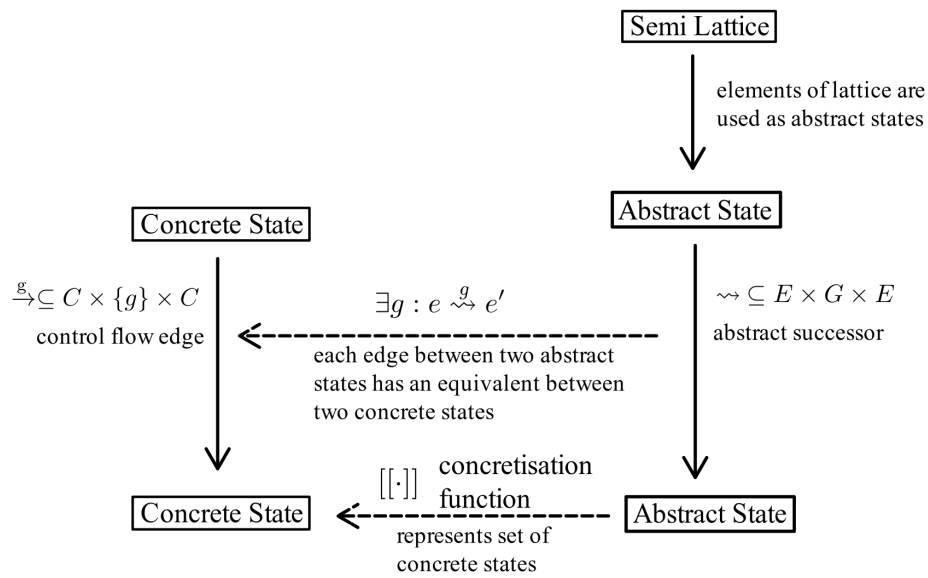

Figure 2.4: Abstract states represent many concrete states. Both share that they are transformed with control-flow edges into new states. Abstract states are derived from the semi-lattice and the control-flow edges are derived from the CFA.

$$(l, v) \sqsubseteq (l', v') \; if \; (l = l' \vee l' = \top_L) \wedge \forall x \in X : v(x) = v'(x) \; \vee \; v(x) = \bot_{\mathbb{Z}} \; \vee \; v'(x) = \top_{\mathbb{Z}}$$

l and l' are assignments of program locations while v and v' are assignments of X where $v(x)$ is the assignment of the variable $x \in X$. Variable assignments in the new state (l', v') must either be $\top$ or the same value as in the state being subsumed.

Examples for Half-order $\sqsubseteq$ on Abstract States Each item in the listing shows two abstract states being compared with $\sqsubseteq$. It is shown if the left state is subsumed by the right state or not.

- $(6, \{x \mapsto 0, y \mapsto 1, z \mapsto 0\}) \sqsubseteq (6, \{x \mapsto 0, y \mapsto 1, z \mapsto 0\})$
- $(5, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\}) \sqsubseteq (5, \{x \mapsto \top, y \mapsto \top, z \mapsto 0\})$
- $(5, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\}) \not\sqsubseteq (6, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\})$
- $(5, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\}) \sqsubseteq (\top, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\})$

2.1.4 Relationship between Concrete and Abstract States in the Value Analysis

Each abstract state can be transformed into the set of concrete states (region) that it represents by applying the concretization function $[[\cdot]] : E \to 2^C$. Regarding the value analysis implementation, this results in all variables that are $\top_Z$ in the abstract state being assigned concrete integer values so that in the end all variables are bound to integers. Similar to the concrete states, abstract states are also transformed by control-flow edges into new abstract states, performed by the transfer relation $\rightsquigarrow \subseteq E \times G \times E$. These resulting states are called *abstract successors*. $e \overset{g}{\rightsquigarrow} e'$ describes an abstract successor state e' reached from e through the control-flow edge g.

Example for Abstract States, Successors and Concretization The listing shows some of the abstract states that are possible when applying the value analysis on the CFA from the example in figure 2.2. Also shown is the successor relation $\rightsquigarrow$ and how abstract states are transformed into concrete states with $[[\cdot]]$.

- $(5, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\})$
  Abstract state subsuming all possible abstract states at program location 5.

- $(5, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\}) \overset{y==1}{\rightsquigarrow} (6, \{x \mapsto 0, y \mapsto 1, z \mapsto 0\})$
  Transition relation from location 5 to 6 highlighting the differences of the possible states and what the value analysis would do. While y could be any value at location 5 it has to be $y \mapsto 1$ at location 6.

- $[[(6, \{x \mapsto 0, y \mapsto 1, z \mapsto 0\})]] = \{(c(pc) = 6, c(y) = 1, c(x) = 0, c(z) = 0)\}$
  Applying the concretisation function to the abstract state of location 6, the resulting set only contains one abstract state, as $y \mapsto 1$ is the only possible assignment for y.

- $[[(5, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\})]] =$
  $\{(c(pc) = 5, c(y) = n, c(x) = 0, c(z) = 0) | n \in \mathbb{N}\}$
  The abstract state of location 5 represents multiple concrete states as $y \mapsto \top$ could be any integer value.

Summarizing the above, figure 2.4 illustrates the interaction between abstract and concrete states. Concrete states are transformed into other concrete states by applying control-flow edges. Similarly, the transfer relation $\rightsquigarrow$ transforms abstract states into their successor abstract states defined by the control-flow edges. Every abstract state can be converted into (a set of) concrete states with the concretisation function $[[\cdot]]$. Note that the figure only shows this function for the lower abstract state.

2.2 Configurable Program Analysis (CPA)

The value analysis is build on the CPA concept, and explores the abstract states of a program in order to generate tests. During this exploration, an abstract reachability graph (ARG) is constructed, consisting of abstract states as nodes and the successor relations as edges. As more of the ARG gets explored, more test goals are reached and more coverage is achieved.

2.2.1 Components of the CPA

A configurable program analysis $\mathbb{D} = (D, \rightsquigarrow, merge, stop)$ on a CFA consists of an *abstract domain* $D = (C, \mathcal{E}, [[\cdot]])$, the transfer relation $\rightsquigarrow$, both components have already been discussed. Also it consists of the *merge* function, which is not relevant to our implementation. Last is the *termination check* stop : $E \times 2^E \rightarrow \mathbb{B}$, that is defined as $stop(e, R) = \exists e' \in R$. It compares an abstract state e with a set of abstract states R using the *half-order* $\sqsubseteq$ of the *semi-lattice* and checks if that state is subsumed by the set. *stop* is used to determine whether a branch in the ARG should be further explored or not.

2.2.2 CPA Algorithm

Simply put, the CPA explores the successors of abstract states in order to reach new program locations. The "explicit-value analysis[1] [...] tracks for each program variable its current value explicitly (like constant propagation, but without join)."[8].

Algorithm 1 defines the exploration of the value analysis, starting from a single abstract state e_0. *waitlist* describes the set of states waiting to be explored. *reached* describes the abstract states that have been found so far. The initial state e_0 assigns $\top$ to each tracked variable in X. This means that at the beginning all variables could be assigned any value. The deeper we go into the program following the control-flow edges and fulfilling the conditions set by the assume operations, the more likely it is that a variable has only one or no concrete value assigned. There are two interesting operations here: On line 6, all possible successors of the current state are created with $e \rightsquigarrow e'$. Each e' must be created by a transformation of e via a control-flow edge as described in section 2.1.4. Depending on the previous state there may be no possible value for a variable in the successor state e'. If a variable cannot be assigned any value, the correct representation would be $\perp$ [2]. In the value analysis the bottom element is never actually assigned. Instead no successor state is found and no state is added to *reached*.

In line 6, the algorithm iterates through all possible successor states of e. For each of these states, the *stop* function determines whether the state is already subsumed by the states in *reached*,

[1] Exlicit-value analysis and value analysis are identical.

[2] As described in the Handbook of Model Checking, every variable assignment that has at least one variable assigned to $\perp$ cannot represent any concrete state.

Algorithm 1 CPA $(\mathbb{D}, e_0)$ algorithm for value analysis

Input: a CPA $\mathbb{D} = (D, \rightsquigarrow, merge, stop)$, an intial abstract state $e_0 \in E$
Output: a set of reachable abstract states
Variables: a set $reached \subseteq E$, a set $waitlist \subseteq E$

```
waitlist := {e_0}
reached := {e_0}
while waitlist ≠ {} do
    choose e from waitlist
    waitlist := waitlist \ {e}
    for each e' with e ⇝ e' do
        if ¬stop(e', reached) then
            waitlist := waitlist ∪ {e'}
            reached := reached ∪ {e'}
        end if
    end for
end while
return reached
```

i.e. whether it has already been explored. If a state has not yet been explored yet, it will be added to *waitlist* and *reached*, and its successors will be explored later in the analysis.

2.2.3 Abstract Reachability Graph (ARG)

After the analysis is finished, the set *reached* is returned, consisting of all abstract states that have been found. An Abstract Reachability Graph(ARG) is a way to visualize in what order the abstract states have been explored by the value analysis. The graph consists of the abstract states as nodes, and control-flow edges between those nodes as edges[3]. Figure 2.5 shows the complete ARG obtained when running the value analysis on the example program from figure 2.2. For each program location, at least one abstract state is found. For program locations 10 and 11, multiple abstract states are produced. Both locations come after the if-statement in the source code and are visited once through the left and once the right path in the CFA, respectively.

2.3 CPAcheckers original Test Case Generation Algotithm (TCGA)

Primarily an verification software, CPAchecker also features the *Test Case Generation Algorithm (TCGA)* that uses the value analysis to generate test suites. In this section we explain CPAchecker's

[3]The control-flow edges of the ARG are already defined by the CFA. They do not need to be stored in a separate structure during the value analysis.

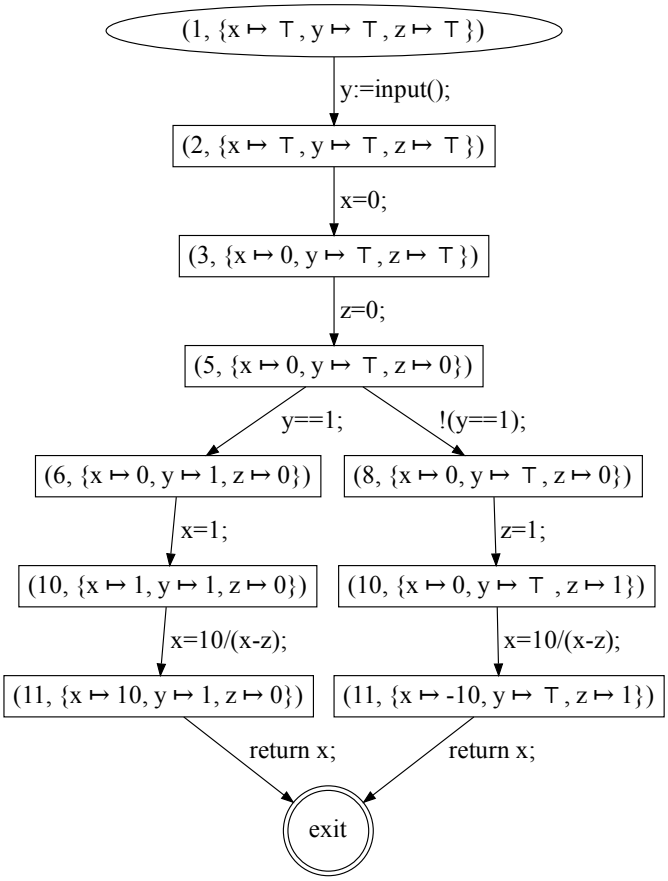

Figure 2.5: Full abtract reachability graph that is the result of the value analysis applied on the c-example from figure 2.2. The eliptic node denotes the start state of the analysis. Each edge displays the control-flow edge, through which a successor state is reached. The ARG branches at the if-statement, producing two subgraphs.

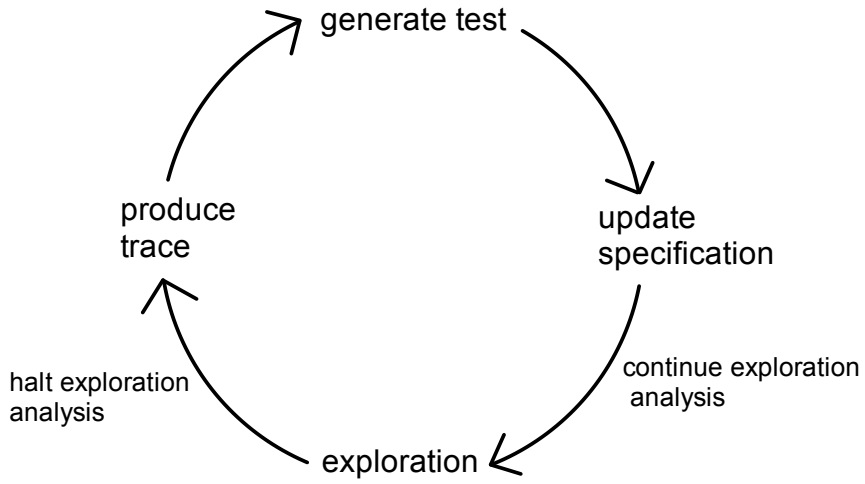

Figure 2.6: Cycle of the TCGA generating tests using the value analysis but without the extractor component.

orignal TCGA and its weakness of featuring no multi-goal coverage, which we introduce with our approach. We based our work on the variant of CPA with constant propagation (value analysis) and without join.

When running the TCGA, the reachability analysis explores the program, trying to reach branches in the CFA (i.e. the test goals). Each time one goal is encountered, a counterexample is created, and from that a test is generated. Then the goal is marked as covered and the analysis resumes.

2.3.1 Steps of the TCGA

With the end goal of generating a test suite, we first have to define *test goals*. Given the structural coverage criteria of branch coverage, *test goals* are the subset of control-flow edges in the CFA that have assume operations. We define a notation to represent an edge between two program locations x and y as $e(x,y)$. The example program from figure 2.2 contains only one if-statement, resulting in two test goals according to the branch coverage criteria. The test goals are the edges $e(5,6)$ and $e(5,8)$.

CPAchecker loops through four steps to generate a test suite: *exploration, produce trace, generate test, update specification*. Figure 2.6 shows this cycle.

1. *exploration.* We refer to the value analysis used here as *exploration analysis*. The exploration analysis expands the ARG by creating new abstract successors. Each time an abstract successor is reached via a control-flow edge that is specified as a test goal, the analysis stops and the state is added to the *reached* set of the CPA algorithm. We will refer to this test goal encountered by the exploration analysis as the *test goal of interest (TGOI)*, as it is in the current focus of the algorithm.

2. *produce trace.* A *counterexample trace* describes a path through the ARG that starts at the root node and its last edge is a test goal[6]. Note that trace and path can be used interchangeably. A path is a route from one program location (see section 2.1.1) to another[4]. We define a *path* as a transition sequence $\pi = \langle n_0, n_1, \ldots, n_k \rangle$, which describes the program locations the path visits, starting at n_0. For example, the sequence $\langle 1, 2, 3 \rangle$ represents a path that begins at location 1, continues to 2, and ends at 3.

   The counterexample trace is obtained from the ARG after reaching the TGOI[5], providing a proof that the TGOI is reachable and giving the fixed path through which it was reached[6]. If no trace can be produced, the reached state is discarded.

3. *generate test.* From the counterexample trace a test is generated and added to the test suite[6]. This test covers the TGOI. A *test vector* or simply test covers a test goal g if its execution results in a program path that reaches g[7]. The test consists inputs for a subset of the variables from the program (see example below).

4. *update specification.* The control-flow edge that is the TGOI is identified and then removed from the set of uncovered test goals Afterwards the exploration analysis continues.

2.3.2 Example of running the TCGA

Figure 2.7 shows the partial ARG for our example program, illustrating at which goals the tests are produced. As there is only one if-statement, there are two test goals, namely the control-flow edges labeled $y == 1$ and $![y == 1]$. From the counterexample trace $\langle 1, 2, 3, 4, 5, 6 \rangle$ the test T1 was generated covering the goal $e(5, 6)$. T2 was generated from the trace $\langle 1, 2, 3, 4, 5, 8 \rangle$, covering $e(5, 8)$. With these two goals achieving full coverage the analysis is finished and no other abstract states are explored. For this reason, the ARG in figure 2.7 lacks some of the abstract states that are present in the full ARG in figure 2.5.

[4] In the context of model checking, a path does not have to stretch from the beginning to the end of a program.
[5] In general, a counterexample trace is produced any time a specification violation is encoutered.
[6] In the context of this thesis, we assume that counterexample traces are always spurious, as tests are never generated from dubious counterexample traces.

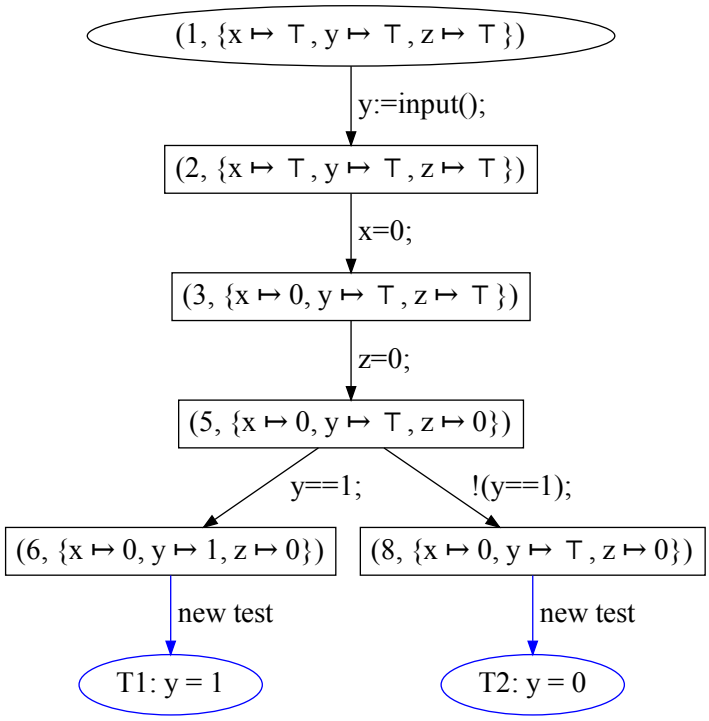

Figure 2.7: ARG that is created by applying the TCGA the example from figure 2.2. The resulting two tests T1 and T2, shown in blue, are created when the two test goals are reached by the value analysis. These two test lead to full coverage.

2.3.3 Problem with CPAchecker's Test Case Generation Algorithm

Collaterally covered goals are the goals a test covers, excluding the TGOI. This collateral coverage is currently overlooked by CPAchecker, which is the problem that we try to overcome with our detection approach. For each test goal encountered, exactly one test is generated, and exactly one test goal is marked as covered. A full cycle of the generation algorithm is run for each goal to generate a new test[7]. As each test has the potential to cover multiple goals, the number of times a new test is generated could be (decisively) smaller than the number of covered test goals. Currently this ratio is 1.

The only path information known about the test is the way from the root node until the TGOI, i.e. the counterexample trace. These are the edges in the CFA that the test definitely covers when executed; the last known edge being the TGOI. Any further edges the test covers after the counterexample trace are not explored. This further execution path will be referred to as the *extended path*.

Example Extended Paths This problem can be illustrated with an extended version of the example from figure 2.2. Recall that two tests are generated for that example and both of them were required for full coverage. That program will be extended: the new version in figure 2.8 includes all the code from the first example, but has a second if-statement in line 11. The two if-statements result in a total of four test goals.

The two tests from figure 2.7 can be reused as we know that they cover goals $e(5,6)$ and $e(5,8)$. When T1 is executed on the program, its extended path is $\langle 6, 10, 11, 12, 16, exit\rangle$, covering goal $e(11,12)$. The extended path of T2 is $\langle 8, 10, 11, 14, 16, exit\rangle$ and covers $e(11,14)$. Without generating a new test, collateral coverage of the remaining goals is achieved. But how can the information about the extended paths be obtained?

[7] Multiple instances of the same test may end up in the test suite and these can be removed. The problem is that those tests had to be generated in the first place.

```
1   int y = input();
2   int x = 0;
3   int z = 0;
4
5   if (y == 1) {
6     x = 1;
7   } else {
8     z = 1;
9   }
10    x = 10 / (x - z);
11    if (x > 0) {
12      x = x - 1;
13    } else {
14      x = x + 1;
15    }
16    return x;
```

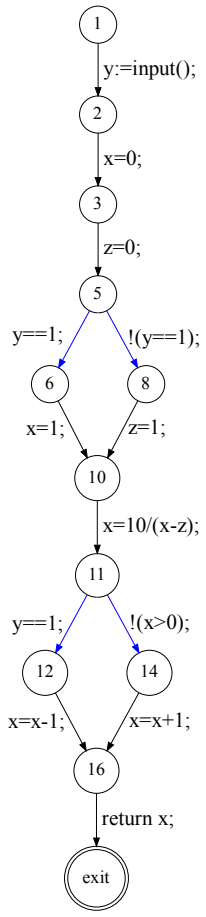

Figure 2.8: Extended example C-program featuring an additonal if-statement and the corresponding CFA. The blue colored edges $e(5,6), e(5,8), e(11,12), e(11,14)$ are test goals.

3 Design of Detection Approach

For our detection approach, we will add a new component called *extractor* to the TCGA (see section 2.3) with the function to discover the extended path (see subsection 2.3.3) and the collaterally covered goals. The general idea of the extractor is to execute the program with the test input starting from the found test goal using explicit-value analysis. We will call this value analysis *extraction analysis*. Like the exploration analysis (see subsection 2.3.1) it is based on the CPA algorithm.

3.1 Main Parts of the Extractor Component

The extractor consists of three steps:

1. *Starting state initialisation*, from where the value analysis begins.
2. The *extraction analysis*, that discovers the extended path.
3. *Coverage update*, marking all newly covered goals.

Figure 3.1 shows the expanded TCGA-loop with the extractor component added. While there is only one exploration analysis that is repeatedly stopped and resumed, each time the extractor is run, a new extraction analysis is started. See section 2.3 for further details on the TCGA we are using as base.

3.1.1 Initialization of the Starting Abstract State

The last abstract state created by the exploration analysis is the state immediately following the TGOI (see subsection 2.3.1). This state will be the starting point for the extraction analysis and needs to be adapted as follows:

- The value analysis stores variable assignments along with the counterexample trace. They represent the values for the current program location, when executing the test on the program. These values are written into the variable assignments of the abstract state, overwriting only assignments of $\top$. Concrete assignments remain unchanged.
- The extraction analysis is initialized with that single state in the waitlist.

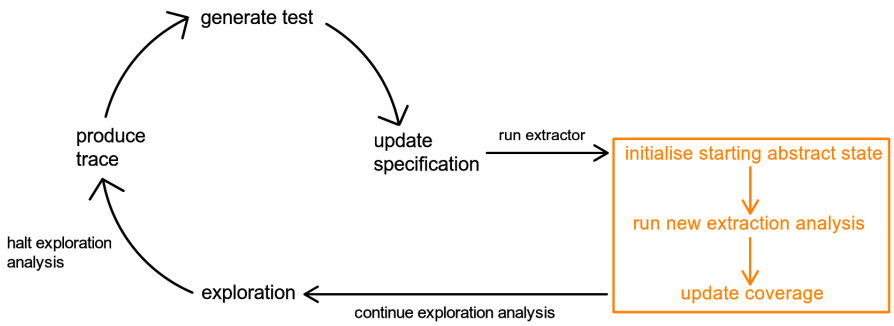

Figure 3.1: Expanded cycle of the TCGA with added extractor component. After a test was generated and the corresponding TGOI was marked as covered, and before the exploration analysis resumes, the extractor (shown in orange) is run. A full extraction analysis is performed and any additional goals will be marked as covered. After that, the exploration analysis continues. While there is only one exploration analysis, each time the extractor is activated, a new extraction analysis is performed.

Example of Initialisation of the Starting State For reference: this example is based on the program from figure 2.8 after generating T2. The full ARGs for the program will be discussed down below in figure 3.2.

- $(8, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\})$
  Last discovered abstract state by the exploration analysis.
- $x = 0, y = 0, z = 0$
  Variable assignments stored alongside the counterexample trace $\langle 1, 2, 3, 5, 8 \rangle$.
- $(8, \{x \mapsto 0, y \mapsto 0, z \mapsto 0\})$
  New initialized abstract state with all tracked variables being assigned a concrete value.

y is overwritten as it is assigned $\top$. x and z state are not changed in the abstract, as they are already assigned concrete values. In this case both x and z were already assigned 0, so it would have made no difference, but even if the values differ, the concrete values in the abstract state are always kept.

3.1.2 Differences between Extraction Analysis and Exploration Analysis

The extraction analysis is a value analysis that determines all abstract states that are reached with certainty when executing the test. The extraction analysis is similar to the exploration analysis used by the TCGA, however there are two, seemingly small, but very important differences that are defined by the following properties:

Continuation Property When encountering an uncovered test goal, the value analysis's default behavior is to stop and return all newly explored abstract states. In the extraction analysis, this stop does not occur. Instead, the analysis continues and ignores the test goal.

Safe-assume Property The successor relation $\rightsquigarrow$ has different behavior regarding assume operations. Assignments of $\top$ are ignored when deciding if the condition is fulfilled and only concrete variable assignments are considered. $\top$ values have the possibility of overestimating variable assignments. They represent that a variable could have any possible value, even though some of these values are impossible in reality. There are cases where despite the occurrence of $\top$, the set of successors is empty. This property guarantees that edges covered by the extended path are always covered when executing the test.

These differences result in the value analysis running until either no more successors can be produced (due to the constraints for assume operations) or the program exit is reached.

Examples of $\rightsquigarrow$ for Extraction Analysis for the Program from Figure 2.8

- $(5, \{x \mapsto 0, y \mapsto \top, z \mapsto 0\}) \overset{y==1}{\rightsquigarrow} \bot$
  Since y is assigned $\top$, it is possible that it is not equal to 1 at this program location. In regard to the safe-assume property, no confident successor can be determined.
- $(5, \{x \mapsto 0, y \mapsto 1, z \mapsto 0\}) \overset{y==1}{\rightsquigarrow} (6, \{x \mapsto 0, y \mapsto 1, z \mapsto 0\})$
  y is assigned exactly the value required by the assume operation, which leads to a successor state that is definitely reachable.

3.1.3 Coverage Update

After running the extraction analysis, the set reached is returned consisting of newly reached abstract states. For each state the ingoing edges are determined and each test goal among them is marked as covered. No tests need to be generated for them as they were covered collaterally.

3.2 Example ARG for Applying Extractor Component

Figure 3.2 shows the multiple ARGs for the program from figure 2.8 generated by the TCGA using both analysis. While the **exploration analysis** creates one contiguous ARG, it is interrupted when T1 and T2 are generated, respectively. The extractor is run for T1 and the **extraction analysis #1** generates a separate ARG with the starting state being based on the last abstract state produced by the **exploration analysis**. It computes the extended path $\langle 6, 10, 11, 12, 16, exit \rangle$ that collaterally covers the goal $e(11, 12)$. After that the **extraction analysis #1** is finished as it

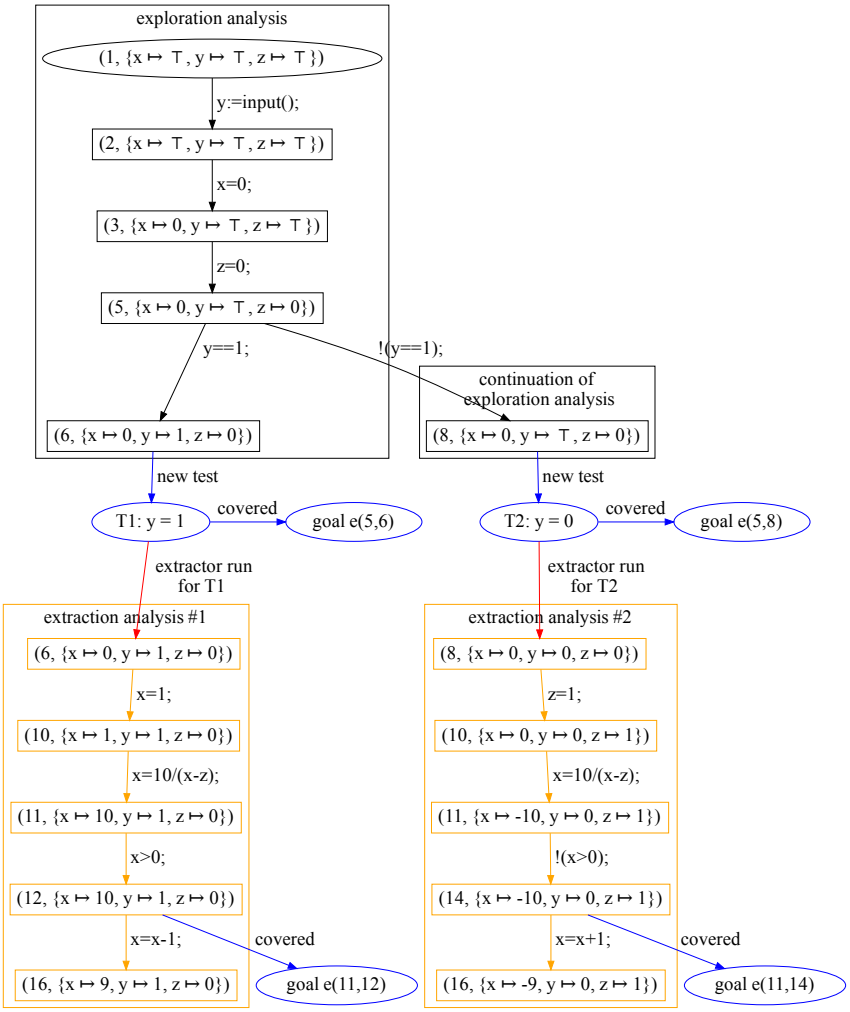

Figure 3.2: Multiple ARGs showing the abstract states created by the TCGA with extractor. The ARG for the extraction analysis is shown in black and the two ARGs from the two extraction analysis are shown in orange. The two test cases and their coverage are shown in blue. The exploration analysis is interrupted by the first extraction analysis, that is run on T1. The extraction analysis finishes and the exploration analysis can continue, generating a second test T2 in the process. A second extraction analysis is run on T2 and full coverage is achieved with requiring only two tests for four goals.

cannot compute successor states anymore. The **exploration analysis** continues expanding its original ARG, generates T2 and stops for a second time. The extractor is run on T2 with the **extraction analysis #2** determining the extended path $\langle 8, 10, 11, 14, 16, exit \rangle$, covering the goal $e(11, 14)$ as a result. The analysis stops as all goals are now covered and the TCGA is finished.

4 Implementation of Extractor in CPAchecker

This first two sections examine how the main parts of the extractor component were implemented in the actual CPAchecker software. The third section shows multiple examples to prove that the implementation achieves the intended goals.

4.1 Realization in CPAchecker's Code

The extractor is an expansion of the TCGA (see section 2.3) and is also realized as part of the already existing *TestCaseGeneratorAlgorithm.java* class. The main method of the extractor is called *runExtractorAlgo()*. All other methods of the extractor are called from here. The extractor itself is called from the *run()* method of the *TestCaseGeneratorAlgorithm.java* class right after a test goal was marked as covered. The reachability analysis done by the extractor is a self contained unit, with the only side effect on the TCGA being the additionally covered goals and the updated specification[1].

We added the following property files to CPAchecker:

- */config/testCaseGeneration-valueAnalysis-NoCegar.properties* Configures a value analysis without counterexample-guided abstraction refinement (CEGAR). This is the basis we used to develop the extractor component and also for comparison in the benchmarking process.
- */config/testCaseGeneration-valueAnalysis-Extractor.properties* Configures a value analysis without CEGAR and with the extractor active. The exploration analysis (see subsection 2.3.1) is configured the same way as in */config/testCaseGeneration-valueAnalysis-NoCegar.properties*.
- */config/includes/testing/extractorAnalysis.properties* Configuration file for the CPAanalysis used by the extractor (extraction analysis, see chapter 3). It must be used with the configuration option *testcase.extractor.config*.

We added the following configuration options:

- *testcase.useExtractor* Is used by *TestCaseGeneratorAlgorithm.java* class. The extractor component will only run as part of the TCGA if this flag is set true.

[1] In practice, test goals are simply removed from the set of uncovered test goals. The specification does not need to be updated at the same time.

- *testcase.extractor.config* Is used by *TestCaseGeneratorAlgorithm.java* class. This option references the configuration file for the extraction analysis, for our implementation we use *extractorAnalysis.properties*.
- *testcase.extractorMode* Is part of *extractorAnalysis.properties* and used by the *TestTargetCPA.java* class. This flag is given to *TestTargetPrecisionAdjustment.java* class to enable the continuation property.
- *cpa.value.extractorMode* Is part of *extractorAnalysis.properties* and used by the *ValueAnalysisTransferRelation.java* class. This flag enables the safe assume property.

4.2 Step by Step Implementation of the Extractor Parts

This section goes over interesting implementation challenges of the three extractor parts *starting state initialization*, *extraction analysis* and *coverage update*, that were introduced in section 3.1.

4.2.1 Initialize Starting Abstract State

The starting abstract state for the extraction analysis is created by cloning the final state discovered during the exploration analysis. A separate ARG is required for the extraction analysis to avoid interference with the data structure of the exploration analysis. The states computed by the extraction analysis cannot be used by the exploration analysis, as they might omit information about variable assignments.

Variable Assignment Representation In CPAchecker's software, the *Valueanalysis state* is part of an abstract state and consists variable assignments. Variables without assignment in the Valueanalysis state are treated as being assigned $\top$. Variables assigned $\bot$ are represented by not creating an abstract state, and in extension no Valueanalysis state.

Initialization of the Starting Abstract State To initialise the starting abstract state for the extraction analysis:

1. The variable values, that are stored together with the counterexample trace, are obtained.
2. If multiple values are stored for a variable, the final value is used.
3. These values are written into the Valueanalysis state of the starting abstract state.
4. Only variables lacking concrete assignments are updated, so only variables that are $\top$ can get a new assignment.

5. Existing assignments of concrete values are preserved unchanged.

The resulting abstract state contains concrete variable assignments of all variables present in the test.

4.2.2 Run Extraction Analysis

The continuation and the safe-assume property (see section 3.1.2) make sure the extractor creates the longest extended path (see subsection 2.3.3) possible with complete confidence. As mentioned above, we introduced two new configuration options to fulfill both properties, resulting in a different behaviour of the CPA algorithm. For the continuation property, the behaviour in the class *TestTargetPrecisionAdjustment.java* is changed. Test goal states are simply ignored when deciding if the program should stop. To fulfill the safe-assume property, certain conditions for generating successors lead to an empty set of successors. This is implemented in the class *ValueAnalysisTransferRelation.java*.

4.2.3 Coverage Update

The first state in the set reached is dropped, as it is the starting state and therefore already covered. States, that are directly after a test goal are automatically marked by the CPA analysis. We dismiss all unmarked states. For the remaining states, the parent state and from that the ingoing control-flow edge are determined[2]. These edges are then removed from the set of uncovered test goals.

4.3 Demonstration of Basic Functionality with Examples

This section demonstrates on three examples, that the new features of the extractor component generally work. It is by no means a complete proof. We demonstrate that the extractor is capable of collaterally covering goals as well as the extraction analysis fulfilling both the safe-assume and the continuation property from section 3.1.2.

4.3.1 Demonstration of Collateral Coverage

For the example in figure 2.8 the following test suite was generated, achieving full coverage and proving that the extractor can collaterally cover goals:

T1 $y = 1$

T2 $y = 0$

[2]As a result of the criterion of branch coverage, these states will have always only one ingoing edge.

Coverage 4/4

4.3.2 Demonstration of Safe-assume Property

```
1   int x = input();
2
3   if (x == 2) {
4     x = 3;
5   } else {
6     x = 4;
7   }
8   int y = input();
9   if (y == 5) {
10    x = 5;
11  } else {
12    x = 6;
13  }
14  return x;
```

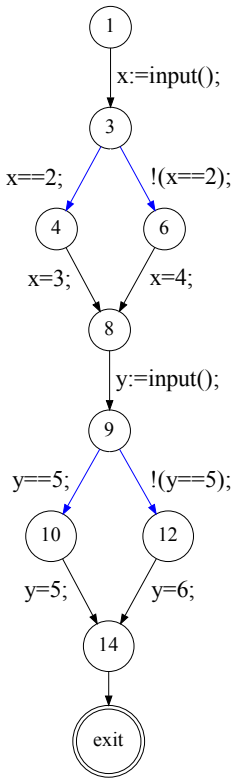

Figure 4.1: Example C-program featuring two if-statements. The blue control-flow edges represent the four test goals. The extraction analysis faces uncertainty when processing the second if-statement regarding the variable y. This is a result of y being only introduced after the first if-statement.

The example program from figure 4.1 has four test goals. The following tests were generated by our TCGA implementation:

T1 $x = 2$

T2 $x = 0$

T3 $x = 2, y = 5$

T4 $x = 2, y = 0$

Coverage 4/4

When generating extended paths, the extraction analysis cannot make a decision at the second if-statement at line 9. y is not part of T1 and T2. Therefore y has the assignment of $\top$ when the extractor is run for these two tests. In line with the safe-assume property, no successor can be made for the assume operation $y == 5$. Therefore the extended paths end at line 9. The remaining two test goals $e(9, 10)$ and $e(9, 12)$ are instead covered by the exploration analysis resulting in the newly generated tests T3 and T4. For this example, no test goals are covered collaterally, proving the safe-assume property holds.

4.3.3 Demonstration of Continuation Property

The program in figure 4.2 features three if-statements and therefore six test goals. The following test suite was created:

T1 $x = 0, y = 1$

T2 $x = 2, y = 0$

T3 $x = 0, y = 0$

Coverage 5/6 (Highest possible coverage as last goal is not reachable)

To cover the test goal $e(4, 7)$, the test T2 is generated. The extended path for T2 is shown in orange in the CFA. The extraction analysis does neither stop at test goals $e(10, 11)$ nor at goal $e(15, 18)$. Instead in continues until the exit. This shows that the continuation property is fulfilled.

```
1   int y = input();
2   int x = input();
3
4   if (y == 1) {
5     return -10;
6   } else {
7     x = x * 2;
8   }
9   y = y + 1;
10  if (x > 0) {
11    x = x - 1;
12  } else {
13    x = x + 1;
14  }
15  if (y == 2) {
16    x = -10;
17  } else {
18    x = 10;
19  }
20  return y;
```

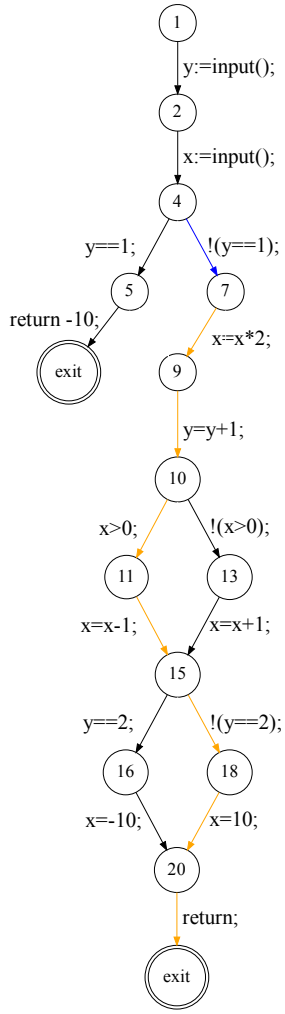

Figure 4.2: Example C-program with three if-statements. After covering the goal $e(4,7)$, shown in blue, an extended path is calculated by the extraction analysis, consisting of the orange control-flow edges. This path covers multiple test goals collaterally. Note that as result of the program exiting after $e(4,5)$, the assumption of $y == 2$ at line 15 can never be fulfilled and the test goal $e(15,16)$ never covered

5 Evaluation of the Extractor

We evaluated our extractor component in regard to the following research questions:
Collateral Coverage How often and to what extent does the extractor cover test goals collaterally, i.e. without generating a new test. *Evaluation Plan:* We compare two variants of CPAchecker: the extractor and the original TCGA (see section 2.3). With both achieving the same coverage, we compare the number of generated tests between both variants. Additionally we measure how many goals were covered by the extractor, the reduction in the size of test suites and how often the extractor made any differences at all in regards to collateral coverage.
CPU-time Consumption When achieving a comparable coverage, how much CPU-time does the test generation of the extractor use compared to the original TCGA. *Evaluation Plan:* We compare CPU-time of the extractor to the original TCGA. Also we measure how many times the extractor took longer than the original TCGA and the average of how much CPU-time was saved.
Memory Consumption While achieving a comparable coverage, how does memory consumption compare between both variants. *Evaluation Plan:* We compare memory of the extractor to the original TCGA, as well as how many times the extractor used more memory and also the average memory reduction.

5.1 Setup

CPAchecker Configurations For our benchmarks we use the value analysis, a CPA, with constant propagation without join. The Extractor was implemented in CPAchecker's TCGA as an expansion and can be enabled via configuration options.

Original TCGA CPAchecker's original TCGA algorithm for creating test suites. As mentioned before, counterexample guided abstraction refinement (CEGAR) is disabled. The CPU-time limit is 900 seconds. *Configuration file*: /config/testCaseGeneration-valueAnalysis-NoCegar.properties

Extractor Expands the original TCGA, adding collateral test goal coverage. It shares all configuration parameters from the original TCGA for the exploration analysis (see subsection 2.3.1). The extraction analysis (see chapter 3) has its own unique configuration file, that is referenced in the configuration below. CEGAR is disabled and it does not check for counterexamples or create them. The extraction analysis has a CPU-time limit of 5 seconds and runs in extractor mode, which behaves differently from the normal value analysis in certain cases. *Configuration file*: /config/testCaseGeneration-valueAnalysis-Extractor.properties

Programs Both the original TCGA and the extractor generate test suites for C-programs. We performed the benchmarks on a set of over 11038 tasks of the TestComp benchmark set[1] using the version with GitHash c28f3cd0f4a93ab3987c2116a6980c3e396c2b14. All tasks are supporting branch coverage.

Resources The machines run on an Intel Xeon E3-1230 CPU with 3.40 GHz frequency. They have 8 processors and 33 GB RAM. The OS is Ubuntu 24.04 with Linux kernel 6.8.0. Each run has the following resources at maximum available: 15 GB of memory, 900 seconds time limit, 4 CPU cores. We used Benchexec[9] version 3.27 for running the benchmarks and enforcing resource limitations.

Availability The data from the benchmarks is published online[2].

5.2 Experiments

First we give an overview about the different status results. A status of "done" means that CPAchecker finished the reachability analysis on its own, either because it created a full test suite, or because the reachability analysis could not find any more successors (with respect to the time limitations imposed by the configuration). There was no interruption due to an error, an exception or reaching resource limits. Other statuses include different types of errors and exceptions, timeout and out of memory.

Out of 11038 tasks, 1342 runs result in either the status "done", "ERROR" or "ERROR (recursion)", with both variants having the same status. Of these, 1195 runs were "done", which is the preferred result. For any other statuses, no data could be obtained; so we can only analyze the results of these 1342 runs. When the benchmarking gets interrupted, because of a timeout, an exception or being out of memory, the process is shut down and no statistics are produced[3].

In total, both variants did not have the same status result for 2247 runs. 31 of these had the status "done" for the original TCGA, with the extractor getting a worse status. But in turn, the extractor achieved the status "done" 104 times while the original TCGA had a worse status. We can say that in general the extractor is more likely to achieve a good status result when the original TCGA failed than vice versa.

Collateral Coverage To evaluate the extractors ability to cover test goals collaterally, Figure 5.1 shows scatterplots that compare the amount of tests generated for the same run by the extractor (x-axis) and the original TCGA (y-axis). These numbers are comparable because both variants

[1] `https://gitlab.com/sosy-lab/benchmarking/sv-benchmarks/`
[2] `https://zenodo.org/records/17619986`
[3] This can also be caused by the status "ERROR (parsing failed)".

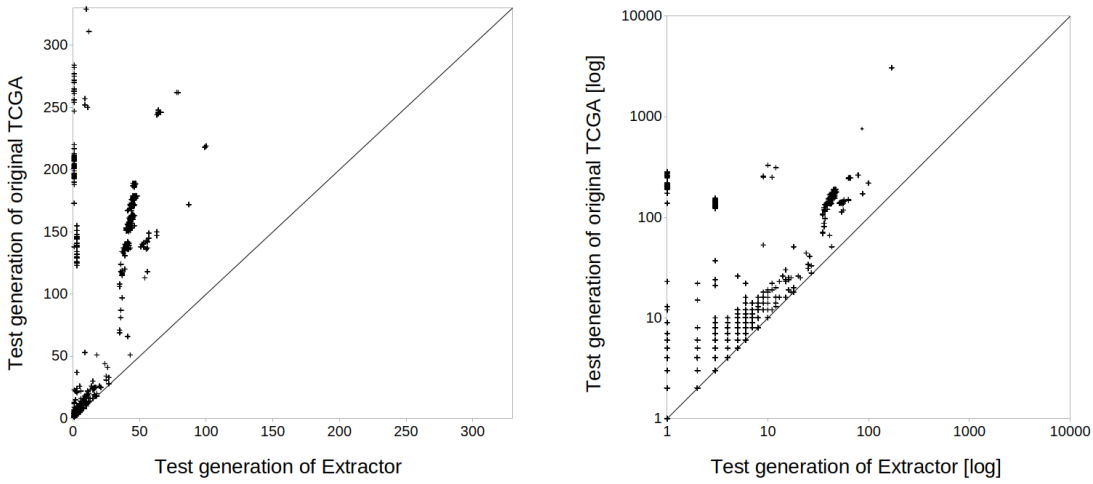

Figure 5.1: Comparing the amount of tests generated by the extractor and by the original TCGA. The left plot has a linear scale and the right plot an logarithmic scale. For each task, both variants achieved the same coverage. If a data point is directly on the diagonal, both variants produced the same amount of tests and there was no collateral coverage.

always achieved the same coverage for a program[4]. Both plots show the same values, the left has a linear scale and the right one a logarithmic scale[5]. We can see that no data points are in the lower right half. This confirms that the extractor never generates more goals than the original TCGA, so it never performs worse in regards to test suite size. 23.9% of the points[6] are on the diagonal, meaning both variants produce the same amount of tests and there are no collaterally covered goals. In the remaining 76.1% of cases, that are all located in the upper left half, there is at least one collaterally covered goal. To achieve the same coverage, the Original TCGA generates 52.0 tests on average, while the extractor generates 11.0 tests on average. The extractor generates 21.1% of the amount of tests the original TCGA required for the same coverage. Overall, the extractor covered 78.9% of all of its covered goals collaterally. The set *ReachSafety-ProductLines* performs particularly well, with many tests achieving a coverage of 100 or more goals. The run *elevator_ spec1_ product31.cil.yml* performs the best, with one test covering 284 test goals, demonstrating how effective collateral coverage can be when applied to a suitable task.

CPU-time Consumption Figure 5.2 shows scatterplots for the CPU-time consumption by the extractor (x-axis) and the original TCGA (y-axis), again in linear scale on the left and logarithmic

[4]In 39 cases, the coverage actually differs, but it does not play a big role. The way all data is calculated is in favor of the original TCGA in regard to this aspect.

[5]One single outlier has been removed in the linear plot.

[6]Here we exclude two cases where both variants covered no test goals at all, and as a result the extractor not being used, rendering a comparison obsolete.

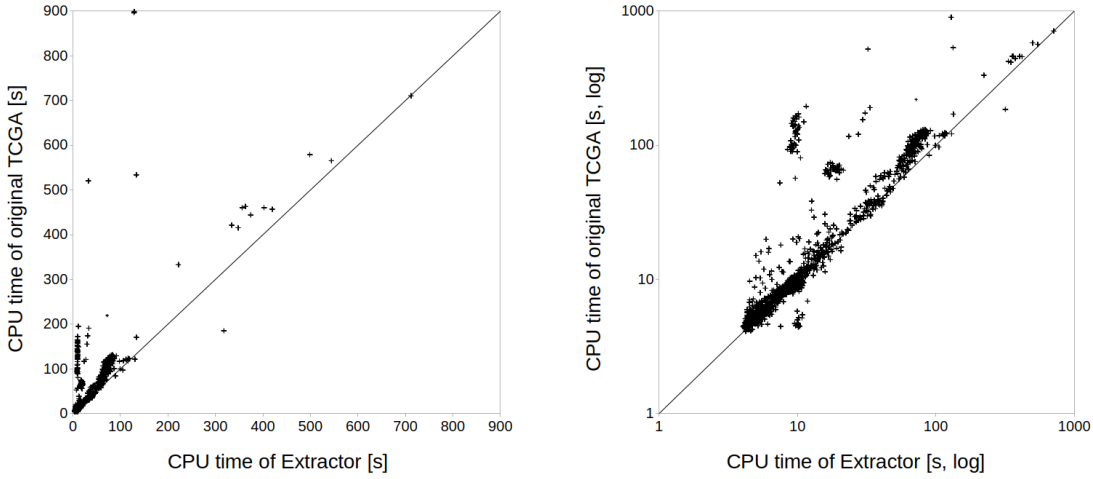

Figure 5.2: CPU-time of the extractor compared to the CPU-time of the original TCGA for achieving the same coverage for a program. The left plot has a linear scale and the right plot an logarithmic scale. Most points are located in the upper left half, showing that the extractor performed better.

scale on the right. The points we see in the upper left half show the results when the extractor is faster than the original TCGA. This is the case for 73.1% of runs. 26.9% of points are in the lower right half, meaning the extractor takes longer in those cases. Nearly half of all points, 54.8%, are within a 10% margin of the diagonal. We can observe most outliers in the upper left half, showing that the extractor performs significantly better for some runs. Overall, CPAchecker uses 87.7% of the CPU-time on average compared to the original TCGA, while achieving the same coverage and generating smaller test suites.

Memory Consumption Figure 5.3 shows the memory used by the extractor (x-axis) and the original TCGA (y-axis) in linear scale (left side) and logarithmic scale (right side). We see that most points are in near proximity of the diagonal, showing that both variants use a similar amount of memory most of the time. To be more precise, for 81.1% of runs, the extractor performed within a 10% margin compared to the original TCGA. We can observe a trend of points in the upper left half, where memory consumption of the extractor was significantly lower. Overall, the average memory consumption of the extractor was 93.3% when compared to the memory use of the original TCGA, so the extractor performed a bit more efficient. For 34.3% of runs the extractor used more memory, performing worse.

Considering only the runs with no collateral coverage, the extractor had a higher CPU-time 63.8% of the time and a higher memory consumption 60.6% of the time compared to the original the original TCGA.

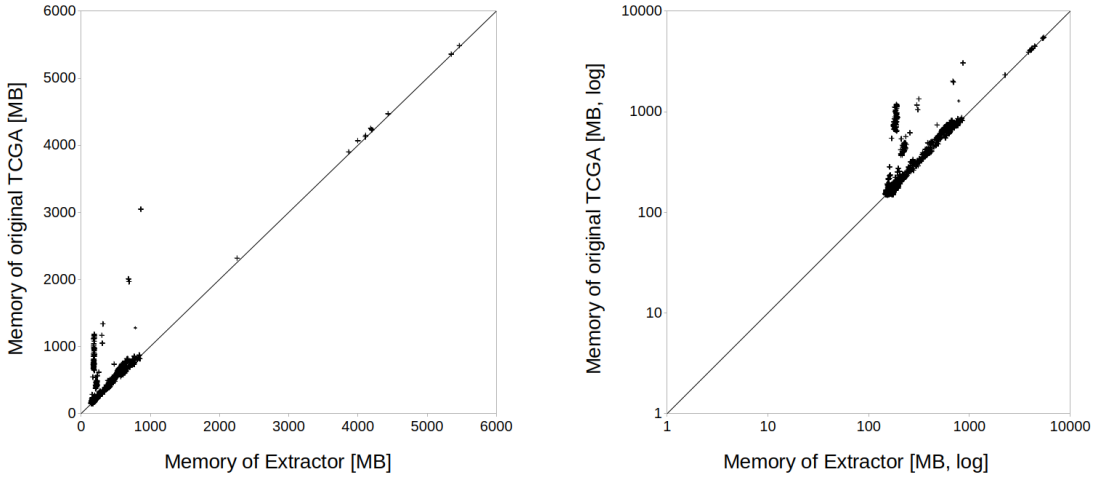

Figure 5.3: Scatterplots showing the memory per run for the extractor and the original TCGA. Left plot is in linear scale and right plot is in logarithmic scale. Most of the time both variants perform similar with the extractor sometimes outperforming the original TCGA.

5.3 Threats to Validity

We only compared the extractor to one other configuration. There is no comparison to more established variants of CPAchecker, such as, value analysis with CEGAR. The extractor component was only tested with benchmark programs. The TestComp benchmarks offer a wide variety of challenges, but in practical tasks, the extractor might perform worse. Trying out other programs and other time limits might not yield results with such a high collateral coverage. The extractor is a new component of CPAchecker and can contain bugs. We tested important properties of the extractor in section 4.3 to counter this. However problems might still surface in the future.

For gathering benchmark statistics we relied on the structures already present in CPAchecker. We did not implement statistics for the extractor itself and measured its side effects on the test generation instead. To our judgement, all relevant information about generated tests and collaterally covered goals can be obtained from the statistics on counterexamples. A big blindspot is, that for a majority of runs the performance of the extractor is unknown, as CPAchecker does not output statistics for these. There were 31 runs where the extractor achieved a worse status than the original TCGA, caused by exceptions. Most of them are part of the *ReachSafety-Floats* set. These exception statuses might be caused by an error in the extractor, which warrants further investigation. When running the benchmarks for the extractor and the original TCGA, we used a different commit version, with the extractor variant running on an earlier version. The two versions differ in that we added a config file specifically for the original TCGA and made one

minor change in the extractor. The latter being, that the *ExpressionValueVisitor* object is obtained in a different way in the *TestCaseGeneratorAlgorithm.java* class. Realistically, this should not make any impact.

6 Related Work

Here we take a look at some inspirations for our approach of covering goals collaterally as well as how other algorithms approach the idea of covering multiple goals simultaneously.

Overview Conditional Testing A conditional tester is a framework for integrating off-the-shelf testers with the goal of generating tests. These conditional testers can also be combined with each other in a variety of ways.

In this framework, generating the test suite and determining test goal coverage are separate actions. The integrated off-the-shelf tester generates a test suite, which may contain only a single test. Then the extractor determines the covered test goals.

6.1 Conditional Testing: Test Goal Extractor

The concept of a test goal extractor[7] is the main inspiration for our approach. A test goal extractor determines the set of test goals that a test suite would cover collaterally. It is a separate component, independent from how the tests are generated and used after test generation. It identifies all test goal locations according to the specification and adds program labels representing the test goals to the code. Then the test suite is executed on the transformed program, and all covered goals are reported. The test goal extractor in conditional testing computes complete extended paths (see subsection 2.3.3) by program execution, compared to the reinitialized reachability analysis in our approach.

In the context of conditional testing being a standalone component, in our implementation the extractor is embedded directly within CPAchecker's *Test Case Generation Algorithm*. This integration allows us to take advantage of the value analysis for computing additional goals, saving the need of executing the program.

An additional difference is that in conditional testing the extractor performs the test execution from the program's beginning, our implementation resumes the analysis, continuing from the current *test goal of interest*.

6.2 Conditional Testing: Cyclic Tester

Conditional testing also includes the variant of a cyclic tester[7]. In this framework, one or more conditional testers are called repeatedly, generating tests, with the result being an increasing set of covered test goals. Most verifiers stop their analysis after finding a counterexample trace and creating a test.

Our solution can be compared to running a cyclic tester, alternating between generating tests with the exploration analysis (see subsection 2.3.1) and determining extra goals with the extraction analysis (see chapter 3). The main difference is that instead of starting from scratch each time, in our approach both components are run as part of the TCGA (see section 2.3), with the benefit of sharing resources.

6.3 Multiple Goal Coverage: CPATiger

CPATiger-MGP[10] is another tool that generates high-quality test cases from counterexample traces of a reachability analysis. MGP stands for multi-goal set partitioning. The test goals are more complex compared to the standard value-analysis, as they can include multiple control-flow edges. Each edge must be reached by the same test case in order for the goal to be covered.

CPA/Tiger-MGP processes multiple test goals at the same time in one reachability analysis. To keep the overhead of the multi-goal reachability information in check, test goals are organised into partitions. For each of these partitions, a reachability analysis is run to cover as many test goals of the partition as possible.

CPA/Tiger-MGP also features a component to cover goals collaterally. Every time a counterexample trace for a goal is found, CPA/Tiger-MGP checks if the path of the counterexample trace covers additional goals from the same and from any other partitions. In contrast to the Extractor-Concept, no extended path information is computed and no coverage is deduced from it.

7 Conclusion

Software testing is a very important area in software development, aiming at assuring software quality and maintainability. An increasing research area is the utilization of software verification tools for automatic test suites generation. The verification software CPAchecker features the built-in Test Case Generation Algorithm (TCGA). It can handle different coverage criterion; among them branch coverage, on which our work is focused. In order to generate tests, CPAchecker uses the value analysis to explore whether test goals are reachable. Every time a test goal is discovered, a counterexample trace is produced, and a test that covers this goal is generated. The central problem to this thesis is, that CPAchecker's original TCGA does not investigate whether that test covers any additional test goals. Instead a new test is generated for each individual goal and the ratio of goals covered per test can never be higher than 1. Our solution to this problem is the extractor component, that introduces multi-goal coverage to CPAchecker. The extractor uses CPAchecker's value analysis to conduct an additional reachability analysis for each test generated. This analysis determines if that test collaterally covers any additional goals. To compute the test's extended execution path efficiently, the extractor does not start at the beginning of the program, but at the program location where the previous analysis left off. We clone the last produced abstract state and initialize it with the variable values stored along the counterexample trace. When running its analysis, the extractor only generates those successor states, that can be confidently reached. This is necessary, as there is no counterexample checking to verify if the new path is valid.

We implemented the extractor as an extension of CPAchecker's TCGA. It can be operated by using either our new configuration files or be directly activated via newly added configuration options. We demonstrated the base properties of the extractor, proving the capability of collateral test goal coverage and its ability to create successors that can be confidently reached.

The extractor is very successful in covering goals collaterally. In our benchmarks, multi-goal coverage was present in 79% of the runs analyzed. Overall, test suite size could be reduced by 79% while maintaining the same coverage. In multiple cases, one test covered over 200 goals instead of just one. Compared to the original TCGA, the average CPU-time and memory consumption could be reduced, but only by 12% and 7%, respectively. For cases where the extractor could not reduce the number of tests required, it performed worse compared to the original TCGA for at least 61% of tasks. Currently the main advantage of the extractor seems to be the reduction of test suite size. The savings in computing resources is acceptable, but can definitely be improved in the future.

The next development goal is to study if the extractor can be integrated with the CEGAR variant of the value analysis. The extractor's performance could be improved by making the code more efficient and trying out different configuration options. Most of our benchmark runs did not yield enough data for our evaluation, due to timeout and exception statuses. For further research, collecting statistical data for these cases using an enhanced benchmark setup could lead to a more refined judgement of the extractors performance. In particular, it would be interesting to investigate 31 cases, where the extractor achieved a worse status than the original TCGA. Analyzing these cases might very well reveal a bug or inefficiency of the extractor.

Bibliography

[1] D. Beyer, A. Chlipala, T. Henzinger, R. Jhala, and R. Majumdar. "Generating tests from counterexamples." In: *Proceedings. 26th International Conference on Software Engineering*. 2004, pp. 326–335. ISBN: 0-7695-2163-0. DOI: 10.1109/ICSE.2004.1317455.

[2] D. Beyer, T. A. Henzinger, and G. Théoduloz. "Configurable Software Verification: Concretizing the Convergence of Model Checking and Program Analysis." In: *Proceedings of the 19th International Conference on Computer Aided Verification (CAV 2007, Berlin, July 3-7)*. LNCS 4590. Springer-Verlag, Heidelberg, 2007, pp. 504–518. ISBN: 978-3-540-73367-6. DOI: 10.1007/978-3-540-73368-3_51.

[3] D. Beyer, T. A. Henzinger, and G. Théoduloz. "Program Analysis with Dynamic Precision Adjustment." In: *Proceedings of the 23rd IEEE/ACM International Conference on Automated Software Engineering (ASE 2008, L'Aquila, September 15-19)*. IEEE Computer Society Press, Los Alamitos (CA), 2008, pp. 29–38. ISBN: 978-1-4244-2187-9. DOI: 10.1109/ASE.2008.13.

[4] D. Beyer, S. Gulwani, and D. Schmidt. "Combining Model Checking and Data-Flow Analysis." In: *Handbook on Model Checking*. Springer, 2018, pp. 493–540. ISBN: 978-3-319-10574-1. DOI: 10.1007/978-3-319-10575-8_16.

[5] D. Beyer, A. Holzer, M. Tautschnig, and H. Veith. "Information Reuse for Multi-goal Reachability Analyses." In: *Proceedings of the 22nd European Symposium on Programming (ESOP 2013, Rome, Italy, March 19-22)*. LNCS 7792. Springer-Verlag, Heidelberg, 2013, pp. 472–491. ISBN: 978-3-642-37035-9. DOI: 10.1007/978-3-642-37036-6_26.

[6] D. Beyer and M.-C. Jakobs. "CoVeriTest: Cooperative Verifier-Based Testing." In: *Proceedings of the 22nd International Conference on Fundamental Approaches to Software Engineering (FASE 2019, Prague, Czech Republic, April 6-11)*. LNCS 11424. Springer, 2019, pp. 389–408. DOI: 10.1007/978-3-030-16722-6_23.

[7] D. Beyer and T. Lemberger. "Conditional Testing - Off-the-Shelf Combination of Test-Case Generators." In: *Proceedings of the 17th International Symposium on Automated Technology for Verification and Analysis (ATVA 2019, Taipei, Taiwan, October 28-31)*. LNCS 11781. Springer, 2019, pp. 189–208. ISBN: 978-3-030-31784-3. DOI: 10.1007/978-3-030-31784-3_11.

[8] D. Beyer and S. Löwe. "Explicit-State Software Model Checking Based on CEGAR and Interpolation." In: *Proceedings of the 16th International Conference on Fundamental Approaches to Software Engineering (FASE 2013, Rome, Italy, March 20-22)*. LNCS 7793. Springer-Verlag, Heidelberg, 2013, pp. 146–162. ISBN: 978-3-642-37056-4. DOI: 10.1007/978-3-642-37057-1_11.

[9] D. Beyer, S. Löwe, and P. Wendler. "Benchmarking and Resource Measurement." In: *Model Checking Software*. Springer International Publishing, Cham, 2015, pp. 160–178. ISBN: 978-3-319-23404-5. DOI: 10.1007/978-3-319-23404-5_12.

[10] S. Ruland, M. Lochau, O. Fehse, and A. Schürr. "CPA/Tiger-MGP: test-goal set partitioning for efficient multi-goal test-suite generation." In: *International Journal on Software Tools for Technology Transfer*. Vol. 23. 6. 2021, pp. 853–856. ISBN: 1433-2787. DOI: 10.1007/s10009-020-00574-z.

[11] W. Visser, C. S. Pundefinedsundefinedreanu, and S. Khurshid. "Test input generation with java PathFinder." In: *Proceedings of the 2004 ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA '04. Boston, Massachusetts, USA: Association for Computing Machinery, 2004, pp. 97–107. ISBN: 1581138202. DOI: 10.1145/1007512.1007526.